

- Group number: 65
- Module: 7
- Emrik Dunvald 020208-5759, ADS
- Elias Samantzis 000715-6631, ADS
- Gusamanel@student.gu.se
- Gusdunvem@student.gu.se
- We hereby declare that we have both actively participated in solving every exercise. All solutions are entirely our own work, without having taken part of other solutions.
- Elias hours spent: 25 hours
- Emrik hours spent: 25 hours

1 Reading and Summary:

GUS was an early computer program created in the 1970s to help researchers understand how computers could have conversations with people. It acted like a travel agent helping users plan trips through flights within California. The system did not have to understand all human knowledge, simply only what was needed for travel planning.

GUS used a structured plan called a frame, which represented all the information it needed like destination, dates, and travellers' attributes. GUS could keep the initiative in conversations and could understand if the user gave extra information. On top of that, GUS was able to ask questions by itself, as it would ask the users questions such as, "What time do you want to leave?" and this made GUS more flexible than many early systems.

One useful feature of GUS was that it could understand short vague answers like "Friday" and figure out the meaning from the conversation. It also remembered what was already discussed, so basically if the user asked "What's the next flight?", GUS would know which flight they meant.

Although GUS was advanced for its time it had limits. GUS could not handle very casual conversations or understand why someone wanted certain flights especially when the user provided reasons to their choices. It was also only useful for travel planning, not general conversation. Still, GUS was important because it showed how computers could combine structured knowledge and flexible reasoning and paved the way for future research as it served as an example of the power of AIs.

2 Implement a simple dialog system:

A)

For our system we decided that the system would handle three different scenarios, these three being restaurant recommendation, weather information and flight booking. We felt that it was easier to work with a real dataset and therefore decided to use three different datasets, each one related to one topic. The system has a main chat bot class which will handle the dialog input and processing of the text. The system also has a Task Manager which handles querying the dataset once we have a valid request extracted from the users prompts. The system works by keyword matching and remembers all the users previous inputs from the same session. It will first determine the main topic, which can be either restaurant, travel or weather, and then start extracting required search parameters such as dates and locations. This also makes it easy to scale the system to handle different topics by adding a new method for handling that topic and a task manager to handle communication with the database.

Some of the more non-trivial features of the system is that it is able to parse dates formatted in a natural way which sometimes lead to issues since the function we used to extract the dates sometimes would mistake normal numbers for dates. Another more advanced feature is the ability to distinguish between destination and departure locations. The way it accomplishes this is by looking for the words to and from and then seeing what comes after those words. Finally, we made sure that the system can handle inputs in different orders or all at the same time to make it more flexible. This is done by analysing everything the user has imputed up until that point and extracting all necessary parts in order to make a request from the backend. The system can also recognize what parameters it is missing in order to make a valid request and can prompt the user to input this.

Weather example:

AI: Hello, what can I help you with today?

Human: what's the temperature like in San Diego?

AI: Please provide a date

Human: 14th of January 2024

AI: The weather in San Diego on the 14th of January 2024 will have a temperature of 10.7

AI: Did this answer your question?

Human: yes

Restaurant example:

AI: Hello what can I help you with today?

Human: I would like to eat some food in New York

AI: Please specify the type of food you are looking for

Human: Asian

AI: In New York, here are some restaurants serving Asian:

- Taiwan Pork Chop House with a rating of 4.8.
- Steam Up (Broadway) (Broadway Ave) with a rating of 2.0.

Travel Example:

AI: Hello what can I help you with today?

Human: I want to travel from Delhi on the 26th of June 2023

AI: Please provide a destination

Human: I want to fly to Mumbai

AI: I found 94 flights going from Delhi to Mumbai on the 26th of June 2023:

[List of Flights]

AI: Would you like me to book any of them or narrow down the search?

Human: I want to fly with Air India

I found 30 flights going from Delhi to Mumbai on the 26th of June 2023:

[List of Flights]

AI: Would you like me to book any of them or narrow down the search?

Human: I want to fly at 20:00

AI: Would you like me to book this flight?

Human: yes

AI: Great! I will book the flight going from Delhi to Mumbai on the 26th of June 2023, departing 20:00 with the airline Air India

AI: your flight number is AI-531

AI: Did this answer your question?

Human: yes

Limitations:

Our system is more flexible than a normal system control flow but still has a number of limitations which makes it quite strict. One drawback that comes with remembering the user's previous history is that it can be difficult if the user decided to change their mind halfway through the process since the system still will remember the previous city or date. If the user at the end says that the system didn't answer it's question then it will reset it's memory but this can also be considered a drawback since you might want the system to remember some of the things you previously said. There are some other obvious issues with our system that we would expect with keyword matching such as not handling errors in the spelling well and not handling implicit meaning in text. We have defined a few synonyms to the main topics but if the users say that they are thinking about taking a vacation then the system cannot connect that to the travel topic.

B)

One big improvement we could make to the system if we had more time would be to switch to a more robust method for extracting the topics of the user input, instead of using keyword matching. This could for example be done with a deep learning model such as BERT which can extract contextual understanding of the user input. Having such an advanced model could also utilize a broader vocabulary to understand the user and pick up on words which are spelled incorrectly. Alternatively, we could use fuzzy matching to mitigate the impact of spelling mistakes and match words which are spelled incorrectly with their correct counterpart.

Another thing we could improve would be to match more patterns in the user input for different topics. This would make the code more complex and might not be desirable but it could allow the chatbot to feel more natural. For example if it can match different dishes with different restaurants for the restaurant topic.

A7

March 11, 2025

```
[80]: import pandas as pd
import nltk
from nltk import word_tokenize, sent_tokenize
from nltk.corpus import wordnet
from dateutil import parser
import re
```

```
[81]: weather_df = pd.read_csv('weather_data.csv')
restaurant_df = pd.read_csv('North America Restaurants.csv', usecols=['name',
    ↪ 'city', 'cuisines', 'weighted_rating_value'])
travel_df = pd.read_csv('goibibo_flights_data.csv', usecols=['flight_date',
    ↪ 'airline', 'flight_num', 'from', 'dep_time', 'to'])

weather_df.columns = ["Location", "Date_Time", "temperature", "humidity",
    ↪ "precepitation", "wind"]
restaurant_df.columns = ['name', 'city', 'cuisines', 'rating']
travel_df.columns = ['date', 'airline', 'flight_num', 'from', 'dep_time', 'to']
travel_df = travel_df.drop_duplicates()
print(travel_df)
```

	date	airline	flight_num	from	dep_time	to
0	26-06-2023	SpiceJet	SG-8709	Delhi	18:55	Mumbai
1	26-06-2023	SpiceJet	SG-8157	Delhi	06:20	Mumbai
2	26-06-2023	AirAsia	I5-764	Delhi	04:25	Mumbai
3	26-06-2023	Vistara	UK-995	Delhi	10:20	Mumbai
4	26-06-2023	Vistara	UK-963	Delhi	08:50	Mumbai
...	...	...	...	...	...	...
299204	22-07-2023	Air India	AI-539	Chennai	17:00	Hyderabad
299205	22-07-2023	Air India	AI-430	Chennai	09:55	Hyderabad
299207	22-07-2023	Air India	AI-440	Chennai	06:10	Hyderabad
299209	22-07-2023	Air India	AI-538	Chennai	20:20	Hyderabad
299217	22-07-2023	Vistara	UK-834	Chennai	17:10	Hyderabad

[117881 rows x 6 columns]

```
[74]: main_topics = {'weather': ['weather', 'temperature', 'cold', 'warm', 'hot'],
    'restaurant': ['food', 'restaurant', 'eat', 'drink', 'hungry'],
    'travel': ['travel', 'drive', 'navigation', 'trip', 'transit']}
```

```

    }
main_topics = pd.DataFrame.from_dict(main_topics)

sub_topics = {
    'weather': ['location', 'date', 'temperature', 'humidity',
↳ 'precipitation', 'wind'],
    'resturant': ['name', 'city', 'cuisines', 'rating'],
    'travel': ['date', 'airline', 'flight_num', 'from', 'dep_time',
↳ 'to']
}
sub_topics = pd.DataFrame.from_dict(sub_topics, orient='index')
sub_topics = sub_topics.T
print(sub_topics)

```

	weather	resturant	travel
0	location	name	date
1	date	city	airline
2	temperature	cuisines	flight_num
3	humidity	rating	from
4	precipitation	None	dep_time
5	wind	None	to

0.0.1 Task Manager for handling database requests

```

[4]: class TaskManager:
    def __init__(self, weather_df=None, resturant_df=None, travel_df=None):
        self.weather_df = weather_df
        self.resturant_df = resturant_df
        self.travel_df = travel_df

    def query_weather(self, location, date, search_parameters):
        search_parameters.remove('location')
        search_parameters.remove('date')
        results = {}
        for parameter in search_parameters:
            try:
                results[parameter] = weather_df.loc[
                    (weather_df['Location'] ==
↳ location) &
                    (pd.
↳ to_datetime(weather_df["Date_Time"]).dt.date == pd.to_datetime(date).date()),
                    parameter # Selecting the
↳ column(s) to return
                ].reset_index(drop=True)

                if not results[parameter].empty:
                    results[parameter] = results[parameter].iloc[0]
            else:

```

```

        results[parameter] = None
    except ValueError:
        results[parameter] = None
    return results

def query_resturant(self, city, cuisines, search_parameters):
    search_parameters.remove('city')
    search_parameters.remove('cuisines')
    results = {}
    for parameter in search_parameters:
        try:
            results[parameter] = resturant_df.loc[
                (resturant_df['city'].str.
↳lower() == city.lower()) &
                (resturant_df['cuisines'].
↳str.contains(cuisines, case=False, na=False)),
                parameter # Selecting the
↳column(s) to return
            ].reset_index(drop=True)

            if not results[parameter].empty:
                results[parameter] = results[parameter].iloc[0]
            else:
                results[parameter] = None
        except ValueError:
            results[parameter] = None
    return results

def query_travel(self, search_parameters):
    date = search_parameters['date']
    df = self.travel_df.loc[pd.to_datetime(self.travel_df["date"],
↳dayfirst=True).dt.date == pd.to_datetime(date).date()]
    search_parameters.pop('date')
    for parameter, value in search_parameters.items():
        df = df if value is None else df[df[parameter] == value]
    return df

```

0.0.2 General utility for parsing text requests

```

[75]: class ChatBoxUtility:
    def __init__(self):
        pass

    def extract_main_topic(self, user_query, main_topic, main_topics):
        # Check for main topic
        for topic in main_topics:
            if user_query.intersection(main_topics[topic].values):
                main_topic = topic

```



```

        return main_topic

    def extract_sub_topics(self, sub_topics, topic, tokens, question_information):
        # Ensure topic is not None
        if topic is None:
            return question_information

        topics = sub_topics.loc[:, topic]

        # Prevents None from being passed to get_synonyms
        for topic in topics:
            if topic is None or not isinstance(topic, str):
                continue

            synonyms = self.get_synonyms(topic)
            if tokens.intersection(synonyms):
                question_information.append(topic)

        return list(set(question_information))

    def get_synonyms(self, word):
        # Error handling for empty sets of synonyms
        if not word:
            return set()

        synonyms = set()
        for syn in wordnet.synsets(word):
            for lemma in syn.lemmas():
                synonyms.add(lemma.name().lower())

        return synonyms

    def extract_city(self, dataset, query):
        if 'city' not in dataset.columns:
            print("'city' column not found in dataset")
            return None

        cities = set(dataset['city'].dropna().str.lower())
        found_city = next((loc for loc in cities if loc in query.lower()), None)
        return found_city

    def extract_date(self, user_input):
        user_input = user_input.lower() # Tokenize input
        try:
            date = parser.parse(user_input, fuzzy=True) # Try parsing each word

```

```

        return date.strftime("%Y-%m-%d") # Convert to standard format
    except ValueError:
        return None

    def extract_location(self, dataset, query):
        locations = set(dataset.loc[:, 'Location'])
        found_location = next((loc for loc in locations if loc in query), None)
        return found_location

    def format_date_natural(self, date_str):
        date_obj = pd.to_datetime(date_str)
        day = date_obj.day
        suffix = "th" if 11 <= day <= 13 else {1: "st", 2: "nd", 3: "rd"}.
        get(day % 10, "th")

        # Format final output
        return f"{day}{suffix} of {date_obj.strftime('%B %Y')}"

    def extract_destination(self, query):
        departure = re.search(r"\b(?:from) ([A-Z] [a-z]+(?:\s[A-Z] [a-z]+)*)",
        query)
        destination = re.search(r"\b(?:to) ([A-Z] [a-z]+(?:\s[A-Z] [a-z]+)*)",
        query)
        if departure: departure = departure.group(1)
        if destination: destination = destination.group(1)

        return departure, destination

    def extract_search_parameter(self, dataset, query, parameter):
        parameters = set(dataset.loc[:, parameter])
        found_parameter = next((loc for loc in parameters if loc in query),
        None)
        return found_parameter

```

0.0.3 Actual class for the chat bot

```

[76]: class ChatBox(ChatBoxUtility):
    def __init__(self, main_topics=None):
        ChatBoxUtility.__init__(self)
        self.main_topics = main_topics
        self.main_topic = None
        self.question_information = []

    def ask(self):
        user_query = input('Hello what can I help you with today?')

```

```

self.Task_Manager = TaskManager(weather_df, resturant_df, travel_df)
date = None
question_answered = False
response = None

while not question_answered:

    # lower case and tokenize question
    tokens = set(word_tokenize(user_query.lower()))

    # extract main topic
    self.main_topic = self.extract_main_topic(tokens, self.main_topic,
↪self.main_topics)

    # extract sub topic
    self.question_information = self.extract_sub_topics(sub_topics,
↪self.main_topic, tokens, self.question_information)

    # send question to specified method depending on what the topic is
    if self.main_topic == 'weather':
        user_query, response = self.weather_questions(user_query, self.
↪question_information)

    elif self.main_topic == 'resturant':
        user_query, response = self.resturant_questions(user_query,
↪self.question_information)

    elif self.main_topic == 'travel':
        user_query, response, date = self.travel_questions(user_query,
↪self.question_information, date)

    else: # If no main topic is found
        user_query = input('Sorry I could not understand your question.
↪I can only handle questions related to weather, travel or resturant booking')
        self.question_information = []
        date = None

    if response: # if we have a response
        print(user_query + '\n')
        confirmation = input(f'Did this answer your question?')
        if set(confirmation.lower().split()).intersection({'yes'}):
            question_answered = True
        else:
            self.question_information = []
            user_query = input('Okay, what would you like to know?')
            self.question_information = []

```

```

        date = None

def weather_questions(self, query=None, sub_topics=None):
    question_answered = False
    required_topics = ['date', 'location']
    optional_topics = {'temperature', 'humidity', 'precipitation', 'wind'}

    location = self.extract_location(weather_df, query)
    date = self.extract_date(query)
    if location: sub_topics.append('location')
    if date: sub_topics.append('date')

    for word in query.split():
        synonyms = self.get_synonyms(word.lower())
        for topic in optional_topics:
            if synonyms.intersection(topic) and topic not in sub_topics:
                sub_topics.append(topic)

    missing_topics = [topic for topic in required_topics if topic not in
↳sub_topics]
    sub_topics = list(set(sub_topics))

    if not missing_topics and len(sub_topics)>2:
        question_answered = True
        answer = self.Task_Manager.query_weather(location, date, sub_topics)
        date = self.format_date_natural(date)
        _, valid_response = next(iter(answer.items()))

        if not valid_response:
            response = f'Sorry but I do not have any information on the
↳weather in {location} on the {date}.'
            return response, question_answered

        response = f'The weather in {location} on the {date} will have '
        for key in answer:
            response += f'a {key} of {answer[key]:.1f} '
            if len(answer) > list(answer.keys()).index(key) + 1:
                response += f'and '
        return response, question_answered

    elif missing_topics:
        question_answered = False
        user_response = input(f'Please provide a {' and a '.
↳join(missing_topics)}')

```

```

        user_response += ' ' + query
        return user_response, question_answered

    else:
        question_answered = False
        user_response = input(f'Sure, what would you like to know about the
↪weather in {location} at this date')
        user_response += ' ' + query
        return user_response, question_answered

def travel_questions(self, query=None, sub_topics=None, existing_date=None):
    question_answered = False
    required_topics = ['date', 'departure', 'destination']
    optional_topics = {'airline', 'flight_num', 'dep_time'}
    question_information = {
        'date': None,
        'from': None,
        'to': None,
        'airline': None,
        'flight_num': None,
        'dep_time': None
    }

    if not existing_date:
        date = self.extract_date(query)
    else:
        date = existing_date
    departure, destination = self.extract_destination(query)
    airline = self.extract_search_parameter(travel_df, query, 'airline')
    flight_num = self.extract_search_parameter(travel_df, query,
↪'flight_num')
    dep_time = self.extract_search_parameter(travel_df, query, 'dep_time')

    if date:
        sub_topics.append('date')
        question_information['date'] = date
        existing_date = date
    if departure:
        sub_topics.append('departure')
        question_information['from'] = departure
    if destination:
        sub_topics.append('destination')
        question_information['to'] = destination
    if airline:
        sub_topics.append(airline)
        question_information['airline'] = airline
    if flight_num:

```

```

        sub_topics.append(flight_num)
        question_information['flight_num'] = flight_num
    if dep_time:
        sub_topics.append(dep_time)
        question_information['dep_time'] = dep_time

    missing_topics = [topic for topic in required_topics if topic not in
↳sub_topics]
    sub_topics = list(set(sub_topics))

    if not missing_topics:
        answer = self.Task_Manager.query_travel(question_information)
        date = self.format_date_natural(date)
        _, valid_response = next(iter(answer.items()))

        if valid_response.empty:
            question_answered = True
            response = f'Sorry but I could not find any flights going from
↳{departure} to {destination} on the {date} with the specified requirements.'
            return response, question_answered, date

        if len(answer)>1:
            response = f'I found {len(answer)} flights going from
↳{departure} to {destination} on the {date}: '
            response += '\n'
            for flight in answer.iterrows():
                response += f'Airline: {flight[1]['airline']}, Flight
↳number: {flight[1]['flight_num']}, Departure time: {flight[1]['dep_time']}'
                response += '\n'
            response += f'Would you like me to book any of them or narrow
↳down the search?'
            user_response = input(response)
            user_response += ' ' + query
            return user_response, question_answered, date

        else:
            if set(query.lower().split()).intersection({'book', 'yes'}):
                question_answered = True
                response = f'Great! I will book the flight going from
↳{departure} to {destination} on the {date}, departing {answer['dep_time'].
↳values[0]} with the airline {answer['airline'].values[0]}'
                response += '\n'
                response += f'your flight number is {answer['flight_num'].
↳values[0]}'
                return response, question_answered, date
            else:

```

```

        response = 'Would you like me to book this flight?'
        user_response = input(response)
        user_response += ' ' + query
        return user_response, question_answered, date

    elif missing_topics:
        question_answered = False
        user_response = input(f'Please provide a {' and a '.
↪join(missing_topics)}')
        user_response += ' ' + query
        return user_response, question_answered, date

def restaurant_questions(self, query=None, sub_topics=None):
    question_answered = False
    required_topics = ['city', 'cuisines']
    optional_topics = ['name', 'rating']

    if sub_topics is None:
        sub_topics = []

    # Extracting list of valid cities
    valid_cities = set(restaurant_df['city'].dropna().str.lower().unique())

    # Extract city from user query
    city = self.extract_city(restaurant_df, query)
    if city:
        city = city.lower()
        if city not in valid_cities:
            print(f" City '{city}' is not in the database.")
            city = None # Reset city

    while not city:
        city = input("Please provide a valid city: ").strip().lower()
        if city in valid_cities:
            break
        print("Invalid city. Please enter a valid city from the database.")
        city = None

    if not city:
        return "No valid city provided. Exiting function.", False

    # Extracting cuisine
    cuisines = None
    while not cuisines:
        cuisines = input("Please specify the type of food you are looking_
↪for: ").strip().lower()

```

```

# Converting user input to set of words
user_cuisine_words = set(cuisines.lower().split())

# Splitting multi cuisine values into separate rows for matchmaking
expanded_df = resturant_df.assign(cuisines=resturant_df['cuisines'].str.
↳lower().str.split(',')).explode('cuisines')

# Filtering restaurants by city
city_restaurants = expanded_df[expanded_df['city'].str.lower() == city]

# Checengk which cuisines match before applying the filter
matching_cuisines = [c for c in city_restaurants['cuisines'].unique()
↳if any(word in c for word in user_cuisine_words)]

# Checking if any word in user input matches any word in dataset
↳cuisines
def cuisine_match(cuisine_entry):
    cuisine_words = set(cuisine_entry.lower().split(", "))
    return bool(user_cuisine_words & cuisine_words)

# Applying the function to match any word in the cuisine column
matching_restaurants = city_restaurants[city_restaurants['cuisines'].
↳apply(cuisine_match)]

if matching_restaurants.empty:
    return f"Sorry, I don't have information on restaurants in {city}
↳serving {cuisines}.", True

response = f"In {city.capitalize()}, here are some restaurants serving
↳{cuisines.capitalize()}:\n"
for _, restaurant in matching_restaurants.head(5).iterrows():
    name = restaurant.get('name', 'Unknown Restaurant')
    rating = restaurant.get('rating', 'No rating available')
    response += f"- {name} with a rating of {rating}.\n"

question_answered = True
return response, question_answered

```

0.0.4 Test weather here

```

[77]: chatbot = ChatBox(main_topics=main_topics)

chatbot.ask()

```


Hello what can I help you with today? whats the temperature like in San Diego
Please provide a date 14th of January 2024

The weather in San Diego on the 14th of January 2024 will have a temperature of 10.7

Did this answer your question? yes

0.0.5 Test resturant here

```
[78]: chatbot = ChatBox(main_topics=main_topics)

chatbot.ask()
```

Hello what can I help you with today? I would like to eat some food in New York
Please specify the type of food you are looking for: Asian

In New york, here are some restaurants serving Asian:

- Taiwan Pork Chop House with a rating of 4.8.
- Steam Up (Broadway) (Broadway Ave) with a rating of 2.0.

Did this answer your question? yes

0.0.6 Test travel here

```
[79]: chatbot = ChatBox(main_topics=main_topics)

chatbot.ask()
```

Hello what can I help you with today? I want to travel from Delhi on the 26th of June 2023

Please provide a destination I want to fly to Mumbai

I found 94 flights going from Delhi to Mumbai on the 26th of June 2023:

Airline: SpiceJet, Flight number: SG-8709, Departure time: 18:55

Airline: SpiceJet, Flight number: SG-8157, Departure time: 06:20

Airline: AirAsia, Flight number: I5-764, Departure time: 04:25

Airline: Vistara, Flight number: UK-995, Departure time: 10:20

Airline: Vistara, Flight number: UK-963, Departure time: 08:50

Airline: Vistara, Flight number: UK-945, Departure time: 11:40

Airline: Vistara, Flight number: UK-927, Departure time: 09:30

Airline: Vistara, Flight number: UK-951, Departure time: 14:20

Airline: GO FIRST, Flight number: G8-334, Departure time: 08:00

Airline: GO FIRST, Flight number: G8-336, Departure time: 14:20

Airline: GO FIRST, Flight number: G8-392, Departure time: 15:00

Airline: GO FIRST, Flight number: G8-338, Departure time: 10:30

Airline: Indigo, Flight number: 6E-5001, Departure time: 07:15

Airline: Indigo, Flight number: 6E-6202, Departure time: 12:00

Airline: Indigo, Flight number: 6E-549, Departure time: 14:20

Airline: Indigo, Flight number: 6E-6278, Departure time: 08:45
Airline: Air India, Flight number: AI-887, Departure time: 07:00
Airline: Air India, Flight number: AI-665, Departure time: 08:00
Airline: AirAsia, Flight number: I5-747, Departure time: 19:00
Airline: GO FIRST, Flight number: G8-266, Departure time: 06:25
Airline: GO FIRST, Flight number: G8-101, Departure time: 05:50
Airline: GO FIRST, Flight number: G8-103, Departure time: 19:25
Airline: Air India, Flight number: AI-441, Departure time: 17:50
Airline: Indigo, Flight number: 6E-5328, Departure time: 09:30
Airline: Vistara, Flight number: UK-933, Departure time: 15:30
Airline: Indigo, Flight number: 6E-2046, Departure time: 17:15
Airline: AirAsia, Flight number: I5-744, Departure time: 08:20
Airline: SpiceJet, Flight number: SG-8169, Departure time: 19:45
Airline: Indigo, Flight number: 6E-5041, Departure time: 18:40
Airline: GO FIRST, Flight number: G8-165, Departure time: 06:05
Airline: Indigo, Flight number: 6E-2373, Departure time: 13:50
Airline: Vistara, Flight number: UK-813, Departure time: 17:35
Airline: Vistara, Flight number: UK-817, Departure time: 16:05
Airline: Vistara, Flight number: UK-819, Departure time: 14:15
Airline: Vistara, Flight number: UK-801, Departure time: 09:05
Airline: Vistara, Flight number: UK-815, Departure time: 08:05
Airline: Air India, Flight number: AI-453, Departure time: 04:55
Airline: SpiceJet, Flight number: SG-2976, Departure time: 19:20
Airline: Air India, Flight number: AI-504, Departure time: 21:35
Airline: Air India, Flight number: AI-502, Departure time: 13:30
Airline: Air India, Flight number: AI-506, Departure time: 09:45
Airline: Air India, Flight number: AI-803, Departure time: 06:10
Airline: Air India, Flight number: AI-479, Departure time: 21:15
Airline: SpiceJet, Flight number: SG-339, Departure time: 17:00
Airline: Vistara, Flight number: UK-955, Departure time: 17:45
Airline: Vistara, Flight number: UK-627, Departure time: 13:20
Airline: AirAsia, Flight number: I5-784, Departure time: 16:05
Airline: Air India, Flight number: AI-9643, Departure time: 19:00
Airline: Air India, Flight number: AI-540, Departure time: 20:40
Airline: Air India, Flight number: AI-429, Departure time: 09:50
Airline: Air India, Flight number: AI-439, Departure time: 06:05
Airline: Air India, Flight number: AI-9645, Departure time: 15:50
Airline: Indigo, Flight number: 6E-2193, Departure time: 14:05
Airline: Indigo, Flight number: 6E-2168, Departure time: 18:00
Airline: Indigo, Flight number: 6E-152, Departure time: 10:15
Airline: Indigo, Flight number: 6E-369, Departure time: 21:50
Airline: Vistara, Flight number: UK-899, Departure time: 14:45
Airline: Air India, Flight number: AI-764, Departure time: 16:55
Airline: Vistara, Flight number: UK-747, Departure time: 06:30
Airline: Vistara, Flight number: UK-809, Departure time: 19:50
Airline: Vistara, Flight number: UK-737, Departure time: 15:55
Airline: Indigo, Flight number: 6E-2338, Departure time: 12:45
Airline: GO FIRST, Flight number: G8-237, Departure time: 12:50

Airline: Vistara, Flight number: UK-871, Departure time: 20:35
Airline: Air India, Flight number: AI-762, Departure time: 20:15
Airline: GO FIRST, Flight number: G8-1404, Departure time: 09:10
Airline: Air India, Flight number: AI-512, Departure time: 15:30
Airline: Air India, Flight number: AI-537, Departure time: 16:15
Airline: Vistara, Flight number: UK-977, Departure time: 19:00
Airline: Indigo, Flight number: 6E-184, Departure time: 09:05
Airline: SpiceJet, Flight number: SG-3002, Departure time: 07:45
Airline: Indigo, Flight number: 6E-2102, Departure time: 07:20
Airline: Air India, Flight number: AI-801, Departure time: 17:05
Airline: Vistara, Flight number: UK-637, Departure time: 17:10
Airline: Vistara, Flight number: UK-835, Departure time: 19:55
Airline: Air India, Flight number: AI-531, Departure time: 20:00
Airline: Vistara, Flight number: UK-705, Departure time: 07:30
Airline: Vistara, Flight number: UK-707, Departure time: 17:30
Airline: Vistara, Flight number: UK-673, Departure time: 13:20
Airline: Air India, Flight number: AI-839, Departure time: 21:25
Airline: Vistara, Flight number: UK-879, Departure time: 17:40
Airline: GO FIRST, Flight number: G8-191, Departure time: 09:15
Airline: Air India, Flight number: AI-767, Departure time: 12:10
Airline: Air India, Flight number: AI-401, Departure time: 06:50
Airline: Air India, Flight number: AI-473, Departure time: 18:40
Airline: GO FIRST, Flight number: G8-213, Departure time: 11:50
Airline: Air India, Flight number: AI-409, Departure time: 10:35
Airline: Vistara, Flight number: UK-837, Departure time: 17:25
Airline: Air India, Flight number: AI-877, Departure time: 11:20
Airline: Air India, Flight number: AI-868, Departure time: 18:00
Airline: Air India, Flight number: AI-624, Departure time: 19:00
Airline: Air India, Flight number: AI-544, Departure time: 17:15
Airline: Vistara, Flight number: UK-985, Departure time: 19:50
Airline: Air India, Flight number: AI-807, Departure time: 17:20
Would you like me to book any of them or narrow down the search? I want to fly with Air India

I found 30 flights going from Delhi to Mumbai on the 26th of June 2023:

Airline: Air India, Flight number: AI-887, Departure time: 07:00
Airline: Air India, Flight number: AI-665, Departure time: 08:00
Airline: Air India, Flight number: AI-441, Departure time: 17:50
Airline: Air India, Flight number: AI-453, Departure time: 04:55
Airline: Air India, Flight number: AI-504, Departure time: 21:35
Airline: Air India, Flight number: AI-502, Departure time: 13:30
Airline: Air India, Flight number: AI-506, Departure time: 09:45
Airline: Air India, Flight number: AI-803, Departure time: 06:10
Airline: Air India, Flight number: AI-479, Departure time: 21:15
Airline: Air India, Flight number: AI-9643, Departure time: 19:00
Airline: Air India, Flight number: AI-540, Departure time: 20:40
Airline: Air India, Flight number: AI-429, Departure time: 09:50
Airline: Air India, Flight number: AI-439, Departure time: 06:05
Airline: Air India, Flight number: AI-9645, Departure time: 15:50

Airline: Air India, Flight number: AI-764, Departure time: 16:55
Airline: Air India, Flight number: AI-762, Departure time: 20:15
Airline: Air India, Flight number: AI-512, Departure time: 15:30
Airline: Air India, Flight number: AI-537, Departure time: 16:15
Airline: Air India, Flight number: AI-801, Departure time: 17:05
Airline: Air India, Flight number: AI-531, Departure time: 20:00
Airline: Air India, Flight number: AI-839, Departure time: 21:25
Airline: Air India, Flight number: AI-767, Departure time: 12:10
Airline: Air India, Flight number: AI-401, Departure time: 06:50
Airline: Air India, Flight number: AI-473, Departure time: 18:40
Airline: Air India, Flight number: AI-409, Departure time: 10:35
Airline: Air India, Flight number: AI-877, Departure time: 11:20
Airline: Air India, Flight number: AI-868, Departure time: 18:00
Airline: Air India, Flight number: AI-624, Departure time: 19:00
Airline: Air India, Flight number: AI-544, Departure time: 17:15
Airline: Air India, Flight number: AI-807, Departure time: 17:20
Would you like me to book any of them or narrow down the search? I want to fly
at 20:00
Would you like me to book this flight? yes
Great! I will book the flight going from Delhi to Mumbai on the 26th of June
2023, departing 20:00 with the airline Air India
your flight number is AI-531

Did this answer your question? yes